# BPMS 1.0 SFU FPGA — Project Methodology
# Presenter Side Notes

*For use during the 30-minute methodology presentation*

## Opening framing: what to highlight first

**Use this before Slide 1 or while Slide 1 is on screen.** The opening should make clear that the deck is about engineering control, not slideware and not an LLM demo.

### Why a proper methodology is needed

- We are converting system-level BPMS/SFU architecture into FPGA architecture, module specs, RTL, verification artefacts, and sign-off evidence.
- The expensive failures are the ones found late: an unclear boundary, missing clock/CDC rule, ambiguous backpressure behaviour, or wrong acceptance criterion discovered during RTL simulation, integration, or hardware bring-up.
- The methodology gives each artefact a consumer, each gate a pass/fail definition, and each open item an owner.

### Why LLM assistance changes the process requirement

- This is our first project executed with LLM tools as part of the workflow: Claude for drafting, ChatGPT for adversarial review, and Claude Code for repo-aware edits/checks.
- LLMs increase drafting speed, but they also increase the risk of polished, plausible, wrong text.
- Therefore the process needs explicit citations, greppable assumptions/TBDs, independent review, minimal diffs, and human-only sign-off.

### Meeting objective

- Leave with methodology alignment, ADR-0005 direction, spec template agreement, review cadence agreement, and named owners for the blocking open items.
- If the six-week schedule is not accepted, capture the exact blocker rather than leaving with vague disagreement.

## One-minute opening script

**Suggested wording:** “The purpose of this meeting is not to report status. It is to agree how we are going to run the SFU FPGA architecture-to-RTL flow. The project needs a proper methodology because we are translating system requirements into FPGA artefacts that several people and tools must consume: architecture docs, module specs, reference models, RTL, verification, and sign-off evidence. If the methodology is weak, ambiguity moves downstream and becomes expensive. This is also the first project where we are explicitly using LLM tools in the engineering flow. That makes the process more important, not less: we need citation discipline, explicit assumptions, independent review, and human sign-off. By the end of this meeting, I want us aligned on the gates, the role boundaries, the architect workflow, and the decisions needed to make the six-week plan credible.”

# Slide-by-slide presenter notes

## Slide 1: BPMS 1.0 SFU FPGA — Project Methodology

**Section**
**Frame**

Primary purpose: Set the framing and meeting objective.

### Highlight

- This is the methodology agreement point, not a status update.
- A proper methodology is needed because architecture errors become expensive once discovered in RTL simulation or hardware.
- This is our first project carried with LLM tools such as Claude and ChatGPT, so the process itself must be formalized: citations, gates, review roles, and human sign-off.

### Suggested emphasis

Open by framing the problem: BPMS 1.0 SFU FPGA is not just an RTL task. We are translating system architecture into FPGA architecture, module specs, RTL, verification, and sign-off evidence. Without a methodology, the work becomes a set of disconnected documents and tool experiments.

### Slide speaker notes from deck

Open with what's at stake: by end of this meeting we want to leave with M1 committed, or pushed back with reasons; ADR-0005 either accepted or with the exact blocker named; and the spec template confirmed. Set the expectation that the bulk of the talk is the architect workflow because that is what we are committing to deliver against.

### Transition / close

Transition: explain why the existing framework was not enough and why an execution layer was added.

## Slide 2: Why now + what is new

**Section**
**Frame**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- The original framework defined what to write; it did not define how to execute.
- The new execution layer makes the schedule credible and aligns with the RTL designer flow.

### Slide speaker notes from deck

Keep this short. The point is to motivate why this is not just a template discussion. The system architect asked for a delivery date; that date needs a methodology and an execution model.

## Slide 3: Two-layer framework + project phases

**Section**
**Methodology**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- Keep the separation between spec layer and execution layer.
- Phases overlap; gates, not calendar dates, control promotion.

### Slide speaker notes from deck

This folder defines what the architecture is; the methodology docs define how it gets built. Resist requests to merge them. The split allows specification to baseline before tooling is finalised. Phases overlap, but gates control promotion.

## Slide 4: Repo layout: flat by artefact type

**Section**
**Methodology**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- Flat by artefact type, not lifecycle phase.
- Reference models stay in the design repo because they are part of the executable specification.

### Slide speaker notes from deck

Three points only: flat by artefact type, design and verification are separate repos, and reference models live with specs because they make bit-exact acceptance enforceable.

## Slide 5: Gate reference card

**Section**
**Methodology**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- This is the reference card for the rest of the talk.
- G1-G3 are architecture gates, G4a is spec sign-off, G4b is designer-owned RTL/uArch sign-off, G5 is DV handoff.

### Slide speaker notes from deck

This slide is intentionally a reference card. It keeps the rest of the deck short: when a later slide says G4a or G5, point back here instead of re-explaining the gate.

### Transition / close

Transition: with the gates defined, move to who owns each class of artefact.

## Slide 6: Three roles, three sets of artefacts

**Section**
**Roles**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- Roles are work-product ownership boundaries, not org-chart boxes.
- One engineer can fill multiple roles, but the artefact ownership still remains explicit.

### Slide speaker notes from deck

Note explicitly: the architect does not own the uArch document. That is a deliberate role boundary, codified in ADR-0005. A single person may fill multiple roles; the boundary is about artefacts.

## Slide 7: The spec/uArch split (ADR-0005)

**Section**
**Roles**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- This is the key governance decision: MS = external contract; uArch = internal implementation.
- Architect owns G4a; RTL designer owns G4b.

### Slide speaker notes from deck

Spend about 90 seconds here. This is the slide everyone needs to remember. The SFU's functional blocks may decompose into multiple SystemVerilog files; the architect does not dictate that decomposition.

## Slide 8: Reference model ownership + why the split matters

**Section**
**Roles**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- For DSP/numerical blocks, the Python reference model is part of the spec.
- Bit-exact correlation makes acceptance objective.

### Slide speaker notes from deck

This is the answer to how bit-exact is made enforceable. The architect signs the refmodel as authoritative. If the designer asks what the block outputs when input is X, it is a module spec question. If the designer asks what the FSM looks like, it is a uArch question.

### Transition / close

Transition: now that role boundaries are clear, walk through the architect workflow in detail.

## Slide 9: Architect workflow at a glance

**Section**
**Workflow**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- The architect workflow is dependency-ordered.
- Core ICDs must be stable before dependent module specs can become design_ready.

### Slide speaker notes from deck

Set the scope before the mechanics. ICDs come between orient sections and contracts/budgets sections because module specs need frozen ICDs and contract sections reference them.

## Slide 10: Section-by-section drafting: markers + citation

**Section**
**Workflow**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- Work section by section, not whole-document by whole-document.
- Markers and citations are the anti-laundering discipline for LLM output.

### Slide speaker notes from deck

The cadence sets the operating tempo: section-sized work units, each closed before moving on. The markers are greppable. Citations are the anti-laundering principle: with LLMs in the pipeline, the failure mode is plausible but wrong text.

## Slide 11: Architecture invariants and functional boundary

**Section**
**Workflow**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- Architecture invariants are non-negotiable unless an ADR explicitly changes them.
- The DCU/SFU boundary is load-bearing: per-beam work remains in DCU unless a sourced ADR says otherwise.

### Slide speaker notes from deck

Concrete example: anything that looks like per-beam Doppler in the SFU is an immediate red flag. The DCU owns it. Moving that boundary requires an ADR with explicit system-level justification.

## Slide 12: Design-ready criteria: the G4a checklist

**Section**
**Workflow**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- Read or paraphrase the completeness test.
- This is the practical definition of design_ready.

### Slide speaker notes from deck

Read the completeness test out loud. This is the line we will use repeatedly over the next 6 weeks. It also protects against scope creep into uArch. Pre-align with Gastón before the meeting; if he disagrees with this test, the methodology shifts.

## Slide 13: What the MS does NOT prescribe + common failure modes

**Section**
**Workflow**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- Prevent spec creep into uArch.
- Make external behaviour precise without dictating internal FSMs or pipelines.

### Slide speaker notes from deck

The last failure mode is the most common for architects coming from RTL. Only boundary fixed-point formats are in the MS. Internal precision is a designer choice unless it observably violates a refmodel-defined acceptance criterion.

## Slide 14: LLM-assisted workflow: roles + pipeline

**Section**
**Workflow**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- Three LLM roles, one human authority.
- LLMs draft, review, and apply accepted edits; they do not decide or sign off.

### Slide speaker notes from deck

Even when an LLM-generated artefact looks polished and complete, no LLM signs off. Steps B and E use LLMs with no rewrite authority. Step D writes to the repo only from accepted review items.

## Slide 15: Live demo: real reviewer issue table

**Section**
**Workflow**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- The reviewer output must be an issue table, not rewritten prose.
- This makes review items traceable to gates and commits.

### Slide speaker notes from deck

This is the demo slide. Pull a real three-row excerpt from an existing review if available. Walk through one row and show how it became a commit. If no real review exists before deck build, run one against the current FAD section and replace this table.

## Slide 16: Architect at G4b + later phases

**Section**
**Workflow**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- Architect involvement does not stop at G4a.
- At G4b, architect reviews only architectural consistency, not implementation style.

### Slide speaker notes from deck

Designer does not reach into the spec; architect does not reach into the implementation. Exception: implementation evidence reveals a missing constraint, which goes back into the MS or an ADR. Architect bandwidth does not drop to zero after Phase 2.

### Transition / close

Transition: after the architect workflow, show how the method becomes executable through make targets and sign-off evidence.

## Slide 17: Execution flow: build entrypoints and gate commands

**Section**
**Execution**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- Every flow stage has a make target, a report, and a matching sign-off criterion.
- Evidence is archived by commit hash.

### Slide speaker notes from deck

This is the how-to-run-the-build layer. Two principles: every stage produces an archived report at a commit hash, and each gate command cross-references the matching pass criteria in the sign-off document. No manual Vivado clicks as sign-off evidence.

## Slide 18: Sign-off criteria: gate map

**Section**

Primary purpose: Support the methodology narrative and prepare the next transition.

**Execution**

### Highlight

- Passing means something concrete at every gate.
- Waivers are explicit; no silent exceptions.

### Slide speaker notes from deck

Walk through one column, e.g. G4a means spec sign-off per module and MS reaches design_ready. Each gate has a dedicated sign-off criteria section. Waivers are explicit and reviewed at the gate they affect.

## Slide 19: CI regression levels + waiver discipline

**Section**
**Execution**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- CI levels make the methodology executable.
- Waivers need expiry triggers or they block integration.

### Slide speaker notes from deck

The CI level grid makes the methodology executable. Each level builds on the previous one. Waiver discipline is the implementation-side version of citation discipline: visible, version-controlled, with expiry.

### Transition / close

Transition: close with schedule, contingencies, and specific asks.

## Slide 20: Schedule

**Section**
**Schedule**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- Six weeks is an LLM-assisted estimate, not a promise independent of contingencies.
- LLMs accelerate drafting and consistency checks, not stakeholder decisions or engineering judgment.

### Slide speaker notes from deck

State the LLM speedup honestly. LLMs accelerate drafting, consistency checks, scaffolding, and format work. They do not speed up engineering thinking, ambiguity resolution, or stakeholder alignment. The six-week estimate assumes all three contingencies hold.

## Slide 21: Decisions to surface and asks per attendee

**Section**
**Schedule**

Primary purpose: Support the methodology narrative and prepare the next transition.

### Highlight

- End with concrete asks and dates.
- Get explicit owner assignment for open items and template confirmation.

### Slide speaker notes from deck

End on the asks. Be specific. The suggested dates assume today's date of 28 April 2026. If the meeting slips, all dates slip together.

### Transition / close

Close: ask for agreement or a named blocker. Do not leave decisions ambiguous.

## Likely questions and short answers

### Is this too much process for a small team?

No. The process is lightweight because it is artefact-driven: each document, gate, and report exists because a downstream consumer needs it.

### Are LLMs signing off engineering work?

No. LLMs draft, review, and apply accepted edits. The responsible engineer signs off.

### Why not merge spec and uArch?

Because they have different owners and different consumers. The MS states external behaviour; the uArch states the internal implementation.

### What happens if the six-week estimate slips?

The slip should be tied to one of the named contingencies: template confirmation, source-doc open items, or architectural surprises.

### What is the practical definition of design_ready?

A senior RTL designer can implement the module from the MS plus referenced ICDs without asking about intended external behaviour.